Cadengo Rodriguez Niels Alexander, Entregable No. 3

Manual Contenedores

Preparando postgreSQL para almacenamiento de metadatos de hive.

Descargamos imagen para contenedor.

#docker.io/dockyuki1992/postgres14.4

```
docker.io/dockyuki1992/postgres14.4     latest                    5c2997150f35  7 months ago  384 MB
```

Creación contenedor postgre.

#podman run -d --privileged --name postgres --add-host postgres:<addr> -e POSTGRES_PASSWORD=test -e POSTGRES_USER=test -p 5432:5432 <container-image-ID>

Acceder al contenedor postgres.

#podman exec -it postgres bash

```
[root@vmrhel-container ~]# podman exec -it postgres bash
root@021582ebf6ea:/#
```

El comando psql ejecuta un front-end basado en la terminal de postgreSQL, e permite escribir consultas de forma interactiva, enviarlas a postgreSQL y ver los resultados de la consulta.

Creamos una base de datos que será la que se hará referencia para el metastore de hive y ahí pueda crear su esquema de datos.

Con el comando “\l” podemos ver todas bases de datos existentes.

```
[root@vmrhel-container ~]# podman exec -it postgres bash
root@021582ebf6ea:/# psql -U test
psql (14.4 (Debian 14.4-1.pgdg110+1))
Type "help" for help.

test=# \l
                              List of databases
    Name     | Owner | Encoding |  Collate   |   Ctype    | Access privileges
-------------+-------+----------+------------+------------+-------------------
 metastoredb | test  | UTF8     | en_US.utf8 | en_US.utf8 |
 postgres    | test  | UTF8     | en_US.utf8 | en_US.utf8 |
 template0   | test  | UTF8     | en_US.utf8 | en_US.utf8 | =c/test          +
             |       |          |            |            | test=CTc/test
 template1   | test  | UTF8     | en_US.utf8 | en_US.utf8 | =c/test          +
             |       |          |            |            | test=CTc/test
 test        | test  | UTF8     | en_US.utf8 | en_US.utf8 |
 test99      | test  | UTF8     | en_US.utf8 | en_US.utf8 |
(6 rows)
```

Con el comando “\c” nos podemos conectar a una base de datos.

Con el comando “\dt” podemos ver la tabla de bases de datos.

```
test=# \c test99;
You are now connected to database "test99" as user "test".
test99=# \dt
                List of relations
 Schema |             Name              | Type  | Owner
--------+-------------------------------+-------+-------
 public | BUCKETING_COLS                | table | test
 public | CDS                           | table | test
 public | COLUMNS_V2                    | table | test
 public | CTLGS                         | table | test
 public | DATABASE_PARAMS               | table | test
 public | DBS                           | table | test
 public | DB_PRIVS                      | table | test
 public | DELEGATION_TOKENS             | table | test
 public | FUNCS                         | table | test
 public | FUNC_RU                       | table | test
 public | GLOBAL_PRIVS                  | table | test
 public | IDXS                          | table | test
 public | INDEX_PARAMS                  | table | test
 public | I_SCHEMA                      | table | test
 public | KEY_CONSTRAINTS               | table | test
 public | MASTER_KEYS                   | table | test
 public | METASTORE_DB_PROPERTIES       | table | test
 public | MV_CREATION_METADATA          | table | test
 public | MV_TABLES_USED                | table | test
 public | NOTIFICATION_LOG              | table | test
 public | NOTIFICATION_SEQUENCE         | table | test
```

Una vez dentro podemos hacer consultas de tipo SQL.

```
test99=# select * from "SDS";
 SD_ID |              INPUT_FORMAT              | IS_COMPRESSED |              LOCATION                | NUM_BUCKETS |
      OUTPUT_FORMAT                        | SERDE_ID | CD_ID | IS_STOREDASSUBDIRECTORIES
-------+----------------------------------------+---------------+--------------------------------------+-------------+-----------------
----------------------------------------+----------+-------+--------------------------
     1 | org.apache.hadoop.mapred.TextInputFormat | f             | s3a://datalake/dbhiveS3/tablehiveS301 |          -1 | org.apache.hadoo
p.hive.ql.io.HiveIgnoreKeyTextOutputFormat |        1 |     1 | f
     2 | org.apache.hadoop.mapred.TextInputFormat | f             | s3a://datalake/dbhiveS3/tablehiveS302 |          -1 | org.apache.hadoo
p.hive.ql.io.HiveIgnoreKeyTextOutputFormat |        2 |     2 | f
(2 rows)
```

Preparando imagen Minio s3.

Imagen contenedor minio.

quay.io/minio/minio

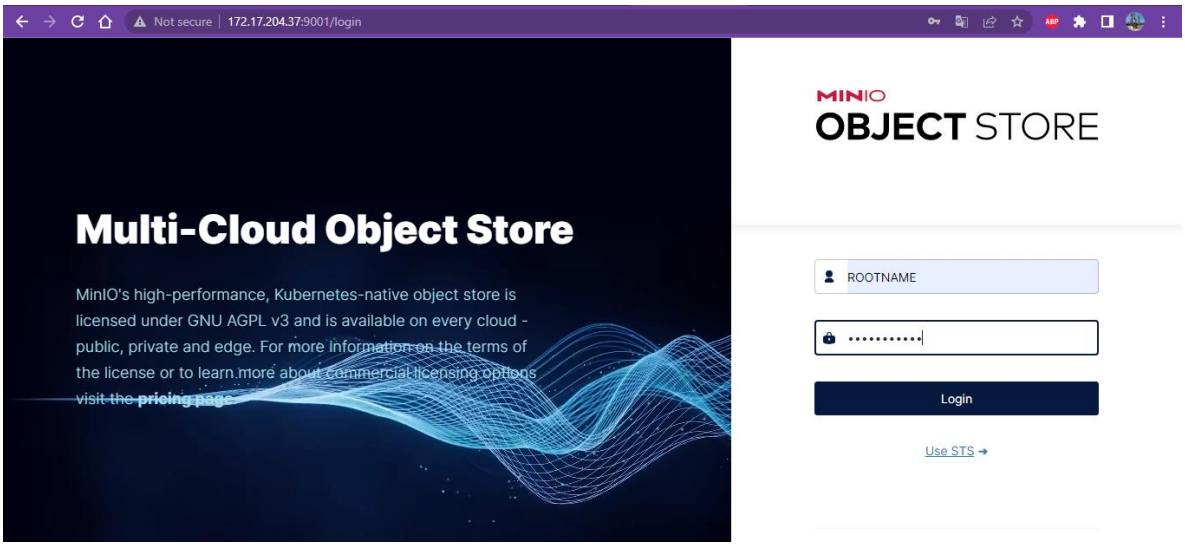
```
quay.io/minio/minio                    latest                    a9a6bbaa87a5   3 weeks ago    237 MB
```

Creación contenedor minio-server.

Creamos una carpeta para asignarle un volumen de almacenamiento llamado /data.

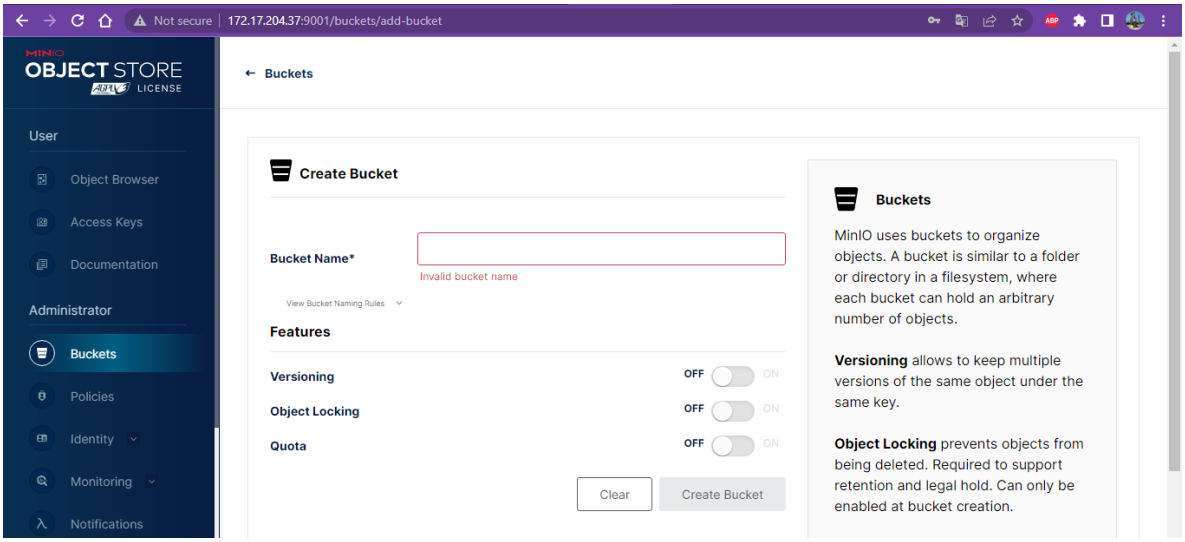
podman run -it --name minio-server --hostname minio-server -p 9000:9000 -p 9001:9001 --privileged -v /residencias/minio/data:/data -e MINIO_REGION="us-east-1" -e MINIO_ROOT_USER="ROOTNAME" -e MINIO_ROOT_PASSWORD="CHANGEME123" minio/minio server /data --console-address ":9001"

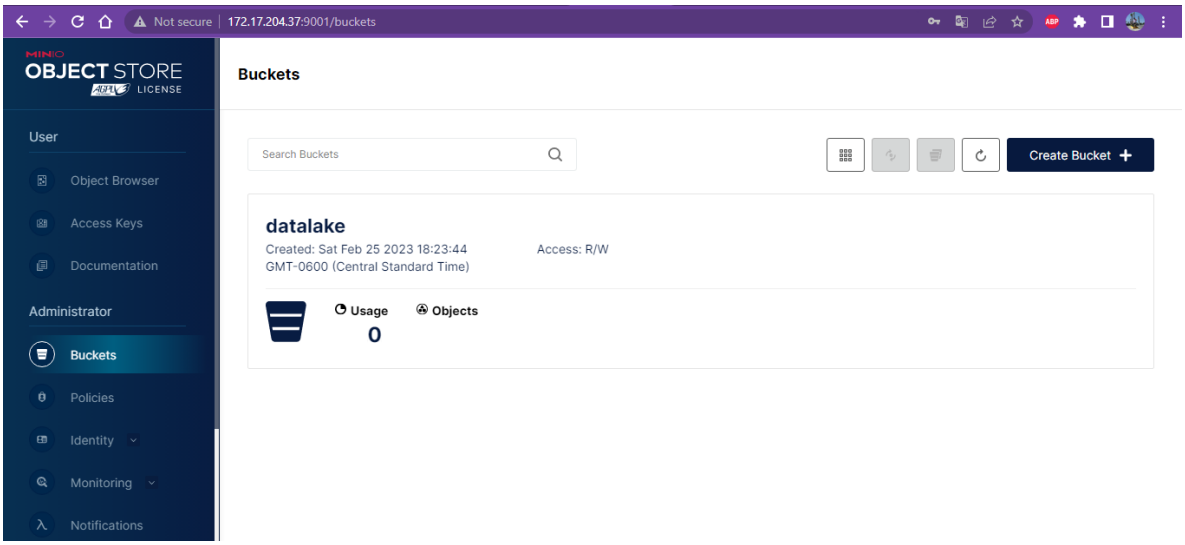
Accedemos a la web por el puerto 9001 para ingresar a la consola minio.



Creamos un bucket de almacenamiento el cual se hará referencia para que hive se conecte y pueda almacenar bases de datos, tablas y/o registros.

En este caso lo llamamos “datalake”.





Preparando Hive metastore y hive server.

Imagen contenedor hive (misma imagen para ambos).

#docker.io/starburstdata/hive

```
docker.io/starburstdata/hive            3.1.2-e.15-x64            9a2ec85906c4   7 months ago   1.42 GB
```

Con esta imagen levantaremos un contenedor hive-metastore y otro hive-server.

Creamos los siguientes archivos con los parámetros de conexión para postgreSQL y Minio en cada contenedor respectivamente.

<u>Hive-Metastore</u>

Dentro de una carpeta hive-metastore crearemos los archivos de configuración para posteriormente hacer el remplazo de archivos agregándolos como volumen.

nano hive-site-metastore.xml

```
<?xml version="1.0"?>

<!-- comentario -->

<configuration>

  <property>

    <name>hive.metastore.local</name>

    <value>false</value>

  </property>

  <property>

    <name>hive.metastore.event.db.notification.api.auth</name>

    <value>false</value>

    <description>

      Should metastore do authorization against database notification related APIs such as
get_next_notification.

      If set to true, then only the superusers in proxy settings have the permission

    </description>

  </property>

  <property>

    <name>hive.metastore.warehouse.dir</name>
```

```
    <value>s3a://basehive/abc3</value>
  </property>
  <property>
    <name>hive.metastore.uris</name>
     <value>thrift://hive-metastore:9083</value>
    <description>Thrift URI for the remote metastore. Used by metastore client to connect to remote metastore.</description>
  </property>
  <property>
    <name>hive.metastore.port</name>
    <value>9083</value>
  </property>
  <property>
    <name>hive.security.authorization.enabled</name>
    <value>false</value>
  </property>
  <property>
    <name>mapreduce.input.fileinputformat.input.dir.recursive</name>
    <value>true</value>
  </property>

<!--  ********************  Configuracion conector Driver postgresql  **********************    -->
  <property>
    <name>javax.jdo.option.ConnectionDriverName</name>
    <value>org.postgresql.Driver</value>
  </property>
  <property>
    <name>javax.jdo.option.ConnectionURL</name>
```

```
    <value>jdbc:postgresql://metastore-postgres:5432/test99</value>
  </property>
  <property>
    <name>javax.jdo.option.ConnectionUserName</name>
    <value>test</value>
  </property>
  <property>
    <name>javax.jdo.option.ConnectionPassword</name>
    <value>test</value>
  </property>


<!--  ********************  Configuracion conector Minio-S3  **********************    -->
<!--
  <property>
    <name>fs.s3n.impl</name>
    <value>org.apache.hadoop.fs.s3a.S3AFileSystem</value>
    <description>The implementation class of the S3A Filesystem</description>
  </property>
-->
  <property>
    <name>fs.s3a.impl</name>
    <value>org.apache.hadoop.fs.s3a.S3AFileSystem</value>
    <description>The implementation class of the S3A Filesystem</description>
  </property>
  <property>
    <name>fs.s3a.connection.ssl.enabled</name>
    <value>false</value>
```

```
    <description>Enables or disables SSL connections to S3.</description>
  </property>
  <property>
    <name>fs.s3a.access.key</name>
    <value>ROOTNAME</value>
  </property>
  <property>
    <name>fs.s3a.secret.key</name>
    <value>CHANGEME123</value>
  </property>
  <property>
    <name>fs.s3a.connection.ssl.enabled</name>
    <value>false</value>
  </property>
  <property>
    <name>fs.s3a.path.style.access</name>
    <value>true</value>
    <description>Enable S3 path style access ie disabling the default virtual hosting behaviour.
            Useful for S3A-compliant storage providers as it removes the need to set up DNS for virtual hosting.
    </description>
  </property>
  <property>
    <name>fs.s3a.endpoint</name>
    <value>http://minio-server:9000</value>
  </property>
  <property>
    <name>fs.s3a.endpoint.region</name>
```

```
    <value>"us-east-1"</value>
  </property>
</configuration>
```

Creamos archivo de configuración core-site.xml para asignar memoria.

```
# nano core-site.xml
<?xml version="1.0"?>
<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>mapreduce.map.memory.mb</name>
    <value>2048</value>
  </property>
  <property>
    <name>mapreduce.reduce.memory.mb</name>
    <value>4096</value>
  </property>
  <property>
    <name>mapreduce.map.java.opts</name>
    <value>-Xmx1638m</value>
  </property>
  <property>
    <name>mapreduce.reduce.java.opts</name>
    <value>-Xmx3278m</value>
  </property>
  <property>
    <name>mapred.map.child.java.opts</name>
```

```
    <value>-Xmx2048m</value>
  </property>
  <property>
    <name>hadoop.proxyuser.hive.groups</name>
    <value>*</value>
  </property>
  <property>
    <name>hadoop.proxyuser.hive.hosts</name>
    <value>*</value>
  </property>


<!--  ********************  Configuracion conector Minio-S3  ********************** -->
  <property>
    <name>fs.s3a.impl</name>
    <value>org.apache.hadoop.fs.s3a.S3AFileSystem</value>
    <description>The implementation class of the S3A Filesystem</description>
  </property>
  <property>
    <name>fs.s3a.connection.ssl.enabled</name>
    <value>false</value>
    <description>Enables or disables SSL connections to S3.</description>
  </property>
  <property>
    <name>fs.s3a.access.key</name>
    <value>ROOTNAME</value>
  </property>
```

```
  <property>
    <name>fs.s3a.secret.key</name>
    <value>CHANGEME123</value>
  </property>
  <property>
    <name>fs.s3a.connection.ssl.enabled</name>
    <value>false</value>
  </property>
  <property>
    <name>fs.s3a.path.style.access</name>
    <value>true</value>
    <description>Enable S3 path style access ie disabling the default virtual hosting behaviour.
            Useful for S3A-compliant storage providers as it removes the need to set up DNS for virtua$
    </description>
  </property>
  <property>
    <name>fs.s3a.endpoint</name>
    <value>http://minio-server:9000</value>
  </property>
  <property>
    <name>fs.s3a.endpoint.region</name>
    <value>"us-east-1"</value>
  </property>
</configuration>
```

Creación contenedor hive-metastore con podman.

podman run -it --name hive-metastore --hostname hive-metastore -p 9083:9083 --privileged -v /residencias/hive-metastore/hive-site-metastore.xml:/opt/apache-hive-3.1.2-bin/conf/hive-site.xml -v /residencias/hive-metastore/core-site.xml:/opt/hadoop-3.3.1/etc/hadoop/core-site.xml -v /residencias/hive-metastore/hosts:/etc/hosts starburstdata/hive:3.1.2-e.15-x64 /bin/bash

Accedemos al contenedor hive-metastore y ejecutaremos en segundo plano el servicio de metastore para iniciar el proceso.

Accedemos a los archivos binarios para poder ejecutar el comando hive.

nohup ./hive --service metastore &

```
[hive@hive-metastore bin]$
[hive@hive-metastore bin]$ cd /opt/apache-hive-3.1.2-bin/bin
[hive@hive-metastore bin]$ nohup ./hive --service metastore &
[1] 45
[hive@hive-metastore bin]$ nohup: ignoring input and appending output to 'nohup.out'
```

Hive-Server

De igual forma dentro de una carpeta hive-server crearemos los archivos de configuración para hive server y posteriormente hacer el remplazo de archivos agregándolos como volumen.

nano hive-site-server.xml

```
<?xml version="1.0"?>

<configuration>

  <property>

    <name>hive.metastore.local</name>

    <value>false</value>

  </property>

 <property>

    <name>hive.metastore.event.db.notification.api.auth</name>

    <value>false</value>

    <description>

      Should metastore do authorization against database notification related APIs such as get_next_notification.

      If set to true, then only the superusers in proxy settings have the permission
```

```
    </description>
  </property>
  <property>
    <name>hive.server2.thrift.port</name>
    <value>10000</value>
  </property>
  <property>
    <name>hive.server2.enable.doAs</name>
    <value>true</value>
  </property>
  <property>
    <name>hive.server2.active.passive.ha.enable</name>
    <value>true</value> # change false to true
  </property>
  <property>
    <name>hive.execution.engine</name>
    <value>mr</value>
  </property>
  <property>
    <name>hive.metastore.uris</name>
     <value>thrift://hive-metastore:9083</value>
    <description>Thrift URI for the remote metastore. Used by metastore client to connect to remote
metastore.</description>
  </property>
  <property>
    <name>hive.security.authorization.enabled</name>
    <value>false</value>
  </property>
```

```
<!--  ********************  Configuracion conector Minio-S3  ********************** -->
  <property>
    <name>fs.s3a.impl</name>
    <value>org.apache.hadoop.fs.s3a.S3AFileSystem</value>
    <description>The implementation class of the S3A Filesystem</description>
  </property>
  <property>
    <name>fs.s3a.connection.ssl.enabled</name>
    <value>false</value>
    <description>Enables or disables SSL connections to S3.</description>
  </property>
  <property>
    <name>fs.s3a.access.key</name>
    <value>ROOTNAME</value>
  </property>
  <property>
    <name>fs.s3a.secret.key</name>
    <value>CHANGEME123</value>
  </property>
  <property>
    <name>fs.s3a.connection.ssl.enabled</name>
    <value>false</value>
  </property>
  <property>
    <name>fs.s3a.path.style.access</name>
    <value>true</value>
```

```
    <description>Enable S3 path style access ie disabling the default virtual hosting behaviour.
            Useful for S3A-compliant storage providers as it removes the need to set up DNS for virtua$
    </description>
  </property>
  <property>
    <name>fs.s3a.endpoint</name>
    <value>http://minio-server:9000</value>
  </property>
  <property>
    <name>fs.s3a.endpoint.region</name>
    <value>"us-east-1"</value>
  </property>
</configuration>
```

nano core-site.xml

<Sera el mismo archivo que se preparó anteriormente >

Creación contenedor hive-server

podman run -it --name hive-server --hostname hive-server -p 10000:10000 -p 10002:10002 --privileged -v /residencias/hive-server/hive-site-server.xml:/opt/apache-hive-3.1.2-bin/conf/hive-site.xml -v /residencias/hive-server/core-site.xml:/opt/hadoop-3.3.1/etc/hadoop/core-site.xml -v /residencias/hive-server/hosts:/etc/hosts starburstdata/hive:3.1.2-e.15-x64 /bin/bash

Accedemos al contenedor hive-server y ejecutaremos en segundo plano el servicio de para iniciar el proceso.

Accedemos a los archivos binarios para poder ejecutar hiveserver2.

nohup ./hive --service hiveserver2 &

```
[hive@hive-server bin]$ nohup ./hive --service hiveserver2 &
[1] 188
[hive@hive-server bin]$ nohup: ignoring input and appending output to 'nohup.out
'
```

Conexión a HiveServer2 mediante Beeline

Apache Beeline es un cliente de Hive que se incluye en los nodos principales del clúster.

./beeline -u jdbc:hive2://hive-server:10000 -n hive -p hive

```
[hive@hive-server bin]$ ./beeline -u jdbc:hive2://hive-server:10000 -n hive -p hive
INIT_MSG: Setting Environment variables for Hive.
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/apache-hive-3.1.2-bin/lib/log4j-slf4j-impl-2.17.2.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop-3.3.1/share/hadoop/common/lib/slf4j-log4j12-1.7.30.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Connecting to jdbc:hive2://hive-server:10000
Connected to: Apache Hive (version 3.1.2)
Driver: Hive JDBC (version 3.1.2)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.2 by Apache Hive
0: jdbc:hive2://hive-server:10000>
```

Creamos base de datos con formato s3 y referencia a “datalake”.

CREATE DATABASE dbhiveS3 LOCATION "s3a://datalake/dbhiveS3";

show databases;

```
0: jdbc:hive2://hive-server:10000> show databases;
INFO  : Compiling command(queryId=hive_20230227191920_9ace2890-742b-4288-a2b1-bb75bf403010): show databases
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Returning Hive schema: Schema(fieldSchemas:[FieldSchema(name:database_name, type:string, comment:from deserializer)], properties:null)
INFO  : Completed compiling command(queryId=hive_20230227191920_9ace2890-742b-4288-a2b1-bb75bf403010); Time taken: 0.018 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20230227191920_9ace2890-742b-4288-a2b1-bb75bf403010): show databases
INFO  : Starting task [Stage-0:DDL] in serial mode
INFO  : Completed executing command(queryId=hive_20230227191920_9ace2890-742b-4288-a2b1-bb75bf403010); Time taken: 0.005 seconds
INFO  : OK
INFO  : Concurrency mode is disabled, not creating a lock manager
+----------------+
| database_name  |
+----------------+
| dbhives3       |
| default        |
+----------------+
2 rows selected (0.053 seconds)
```

Creamos tabla con formato s3 y referencia a “datalake”.

use dbhives3;

CREATE TABLE tablehiveS302 (nombre STRING, numero INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION "s3a://datalake/dbhiveS3/tablehiveS302";

show tables;

```
0: jdbc:hive2://hive-server:10000> show tables;
INFO  : Compiling command(queryId=hive_20230227202917_3db2f2e0-b134-42b0-b66d-b6505fb0d5dd): show tables
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Returning Hive schema: Schema(fieldSchemas:[FieldSchema(name:tab_name, type:string, comment:from deserializer)], properties:null)
INFO  : Completed compiling command(queryId=hive_20230227202917_3db2f2e0-b134-42b0-b66d-b6505fb0d5dd); Time taken: 0.022 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20230227202917_3db2f2e0-b134-42b0-b66d-b6505fb0d5dd): show tables
INFO  : Starting task [Stage-0:DDL] in serial mode
INFO  : Completed executing command(queryId=hive_20230227202917_3db2f2e0-b134-42b0-b66d-b6505fb0d5dd); Time taken: 0.016 seconds
INFO  : OK
INFO  : Concurrency mode is disabled, not creating a lock manager
+----------------+
|    tab_name    |
+----------------+
| tablehives302  |
+----------------+
```

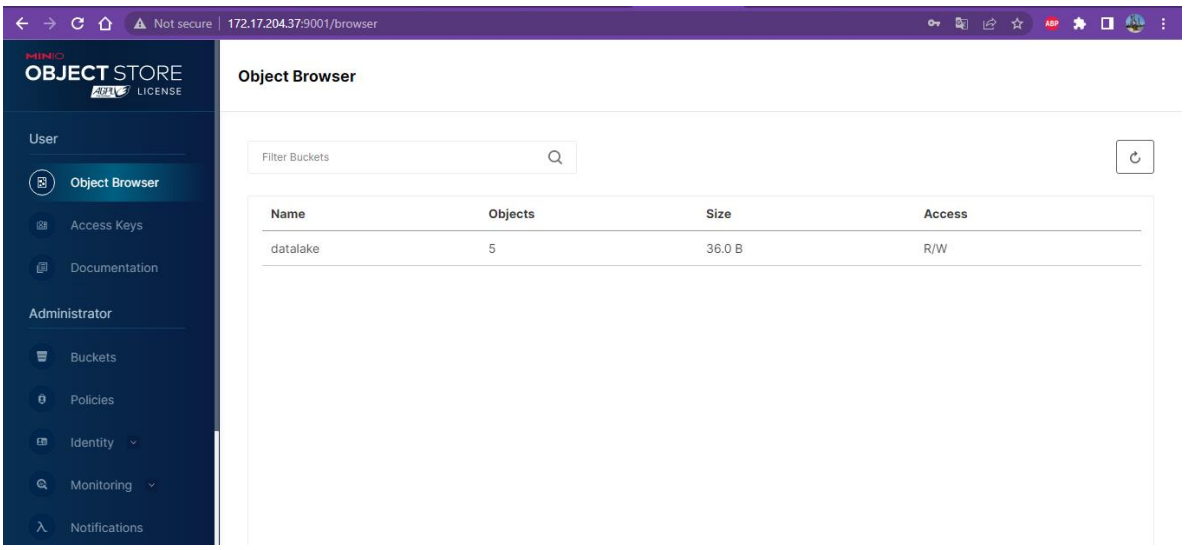
Insertamos registros dentro de la tabla.

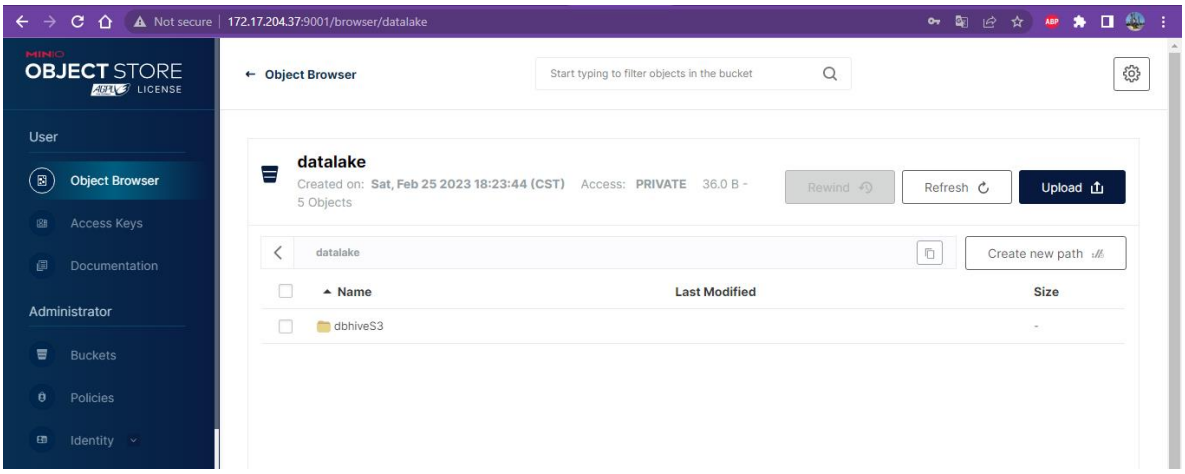
insert into table tablehives302 values ("Admin", 0);

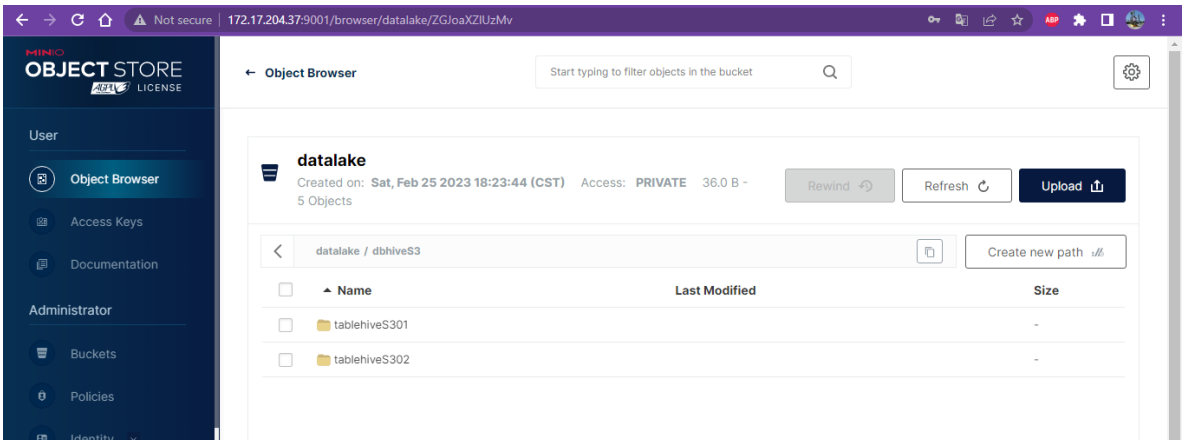
```
0: jdbc:hive2://hive-server:10000> select * from tablehives302;
INFO  : Compiling command(queryId=hive_20230227180102_e3b98dd6-dff9-4761-bf21-016959d51f15): select * from tablehives302
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Returning Hive schema: Schema(fieldSchemas:[FieldSchema(name:tablehives302.nombre, type:string, comment:null), FieldSchema(name:tablehives
302.numero, type:int, comment:null)], properties:null)
INFO  : Completed compiling command(queryId=hive_20230227180102_e3b98dd6-dff9-4761-bf21-016959d51f15); Time taken: 0.138 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20230227180102_e3b98dd6-dff9-4761-bf21-016959d51f15): select * from tablehives302
INFO  : Completed executing command(queryId=hive_20230227180102_e3b98dd6-dff9-4761-bf21-016959d51f15); Time taken: 0.0 seconds
INFO  : OK
INFO  : Concurrency mode is disabled, not creating a lock manager
+----------------------+----------------------+
| tablehives302.nombre | tablehives302.numero |
+----------------------+----------------------+
| Niels                | 1                    |
| Malcom               | 2                    |
| Gillbran             | 3                    |
| Admin                | 0                    |
+----------------------+----------------------+
```

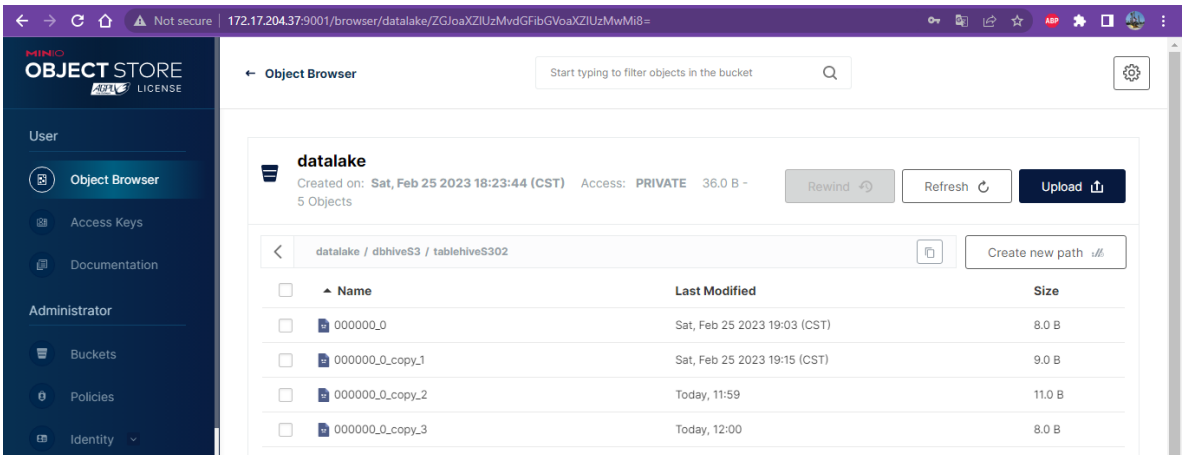
Ejemplo de la base de datos, tablas y registros creados.

Estos ejemplos de datos se vieron reflejados dentro del bucket “datalake” de minio y en el metastore postgresql también pudimos ver que se crearon registros de metadatos con el formato s3 de minio.









Dentro del esquema creado por el metastore de hive en postgresql se crearon las referencias de los metadatos.

Conectamos a la base de datos del metastore, en este caso el ejemplo se llama “test99”.

\c test99;

select * from "SDS";

```
test99=# select * from "SDS";
 SD_ID |              INPUT_FORMAT              | IS_COMPRESSED |              LOCATION              | NUM_BUCKETS |
      OUTPUT_FORMAT                        | SERDE_ID | CD_ID | IS_STOREDASSUBDIRECTORIES
-------+----------------------------------------+---------------+------------------------------------+-------------+-----------------
----------------------------------------+----------+-------+--------------------------
     1 | org.apache.hadoop.mapred.TextInputFormat | f             | s3a://datalake/dbhiveS3/tablehiveS301 |          -1 | org.apache.hadoo
p.hive.ql.io.HiveIgnoreKeyTextOutputFormat |        1 |     1 | f
     2 | org.apache.hadoop.mapred.TextInputFormat | f             | s3a://datalake/dbhiveS3/tablehiveS302 |          -1 | org.apache.hadoo
p.hive.ql.io.HiveIgnoreKeyTextOutputFormat |        2 |     2 | f
(2 rows)
```

select * from "DBS";

```
test99=# select * from "DBS";
 DB_ID |          DESC          |      DB_LOCATION_URI      |   NAME   | OWNER_NAME | OWNER_TYPE | CTLG_NAME
-------+------------------------+---------------------------+----------+------------+------------+-----------
     1 | Default Hive database | s3a://basehive/abc3       | default  | public     | ROLE       | hive
     6 |                        | s3a://datalake/dbhiveS3 | dbhives3 | hive       | USER       | hive
(2 rows)
```

select * from "TBLS";

```
test99=# select * from "TBLS";
 TBL_ID | CREATE_TIME | DB_ID | LAST_ACCESS_TIME | OWNER | OWNER_TYPE | RETENTION | SD_ID |   TBL_NAME    |   TBL_TYPE    |
TEXT | VIEW_ORIGINAL_TEXT | IS_REWRITE_ENABLED
--------+-------------+-------+------------------+-------+------------+-----------+-------+---------------+---------------+-
-----+--------------------+--------------------
      1 |  1677372624 |     1 |                0 | hive  | USER       |         0 |     1 | tablehives301 | MANAGED_TABLE |
     |                    | f
      2 |  1677373049 |     6 |                0 | hive  | USER       |         0 |     2 | tablehives302 | MANAGED_TABLE |
     |                    | f
(2 rows)
```

Apache Atlas

Descargamos imagen contenedor Apache Atlas

#podman pull docker.io/wbaa/rokku-dev-apache-atlas

```
docker.io/wbaa/rokku-dev-apache-atlas  latest          29878d3a3598  2 years ago   2.39 GB
```

Creación contenedor

#podman run -it -d --name atlas-server -p 21000:21000 docker.io/wbaa/rokku-dev-apache-atlas

Modificamos archivo apache-application.properties y agregamos la configuración de hive

#Hive

#atlas.hook.hive.synchronous=false

#atlas.hook.hive.numRetries=3

#atlas.hook.hive.queueSize=10000

#atlas.cluster.name=primary

Modificamos los archivos hive-site.xml y agregamos la siguiente propiedad.

```
<property>
    <name>hive.exec.post.hooks</name>
    <value>org.apache.atlas.hive.hook.HiveHook</value>
  </property>
```

Creamos variables de ambiente y agregamos al path

```
#export JAVA_HOME=/opt/java-openjdk-1.8.0.252/
#export PATH=$PATH:$JAVA_HOME/bin
#export MAVEN_HOME=/opt/apache-maven-3.9.0
#export PATH=$PATH:$MAVEN_HOME/bin
#export M2_HOME=/opt/apache-maven-3.9.0
#export MAVEN_OPTS="-Xms2g -Xmx2g"
#export PATH=$PATH:$M2_HOME/bin
```